Named character escapes

Document Number: P1097R0
Date: 2018-06-21
Audience: SG16, EWG, CWG
Author: R. Martinho Fernandes
Reply-to: cpp@rmf.io (mailto:cpp@rmf.io)

Abstract

This paper proposes a new character escape sequence that enables the programmer to refer to a character by its name or alias.

Motivation

Currently C++ provides four ways to specify characters in string literals:

1. the character itself in the appropriate source encoding;
2. the character as a sequence of `\x` escapes that corresponds to the underlying code units of the character;
3. the character as a `\u` escape that corresponds to the ISO 10646 short identifier of the character;
4. the character as a `\U` escape that corresponds to the ISO 10646 short identifier of the character;

These last two are the best way to represent specific Unicode characters that may be hard to type, hard to read, or that cannot be represented in the source encoding. For example, if one wants a string literal with a combining acute accent, typing `"\u0301"` is easier than producing a combining acute without a base character, and it is guaranteed that it won't render wonky in a text editor (e.g. the acute could be rendered on top of the opening double quotes, making it ugly, and above all, easy to miss).

However, `"\u0301"` is hard to read as well: yes, it's unambiguously clear that this is the character U+0301, but in general there is no way to know what character that is without first looking it up on some table. This solves the problem of clarity that is inherent with typing the character directly, but in turn it obfuscates the actual character behind a series of meaningless numbers.

Other programming languages, in particular Python 3, Perl 5, and Perl 6, have solved this obfuscation problem by providing an alternative character escape mechanism. In these languages, one can specify a character by using its Unicode Name, as in the following Python 3 example:

```
>>> print('A\N{COMBINING ACUTE ACCENT}')
Á
```

Having this ability enables one to choose between the brevity of "0301" and the clarity of "combining acute accent" as desired.

What follows is a Tony Table that demonstrates the differences between the existing mechanisms and the proposed one.

Raw	\x	\u	\U	\N
u8""	u8"\xE2\x80\x8B"	u8"\u200B"	u8"\U0000200B"	u8"\N{ZERO WIDTH SPACE}"
u8""	u8"\xE2\x80\x8B"	u8"\u200B"	u8"\U0000200B"	u8"\N{ZERO WIDTH NON-JOINER}"
u8""	u8"\xE2\x80\x8B"	u8"\u200B"	u8"\U0000200B"	u8"\N{ZERO WIDTH JOINER}"
u8""	u8"\xE2\x81\xA2"	u8"\u2062"	u8"\U00002062"	u8"\N{INVISIBLE TIMES}"
u8"´"	u8"\xC2\xB4"	u8"\u00B4"	u8"\U000000B4"	u8"\N{ACUTE ACCENT}"
u8"́"	u8"\xCC\x81"	u8"\u0301"	u8"\U00000301"	u8"\N{COMBINING ACUTE ACCENT}"
u8"΄"	u8"\xE1\xBF\xBD"	u8"\u1FFD"	u8"\U00001FFD"	u8"\N{GREEK OXIA}"
u8";"	u8"\x3B"	u8"\u003B"	u8"\U0000003B"	u8"\N{SEMICOLON}"
u8";"	u8"\xCD\xBE"	u8"\u037E"	u8"\U0000037E"	u8"\N{GREEK QUESTION MARK}"
u8"Ω"	u8"\xCE\xA9"	u8"\u03A9"	u8"\U000003A9"	u8"\N{GREEK CAPITAL LETTER OMEGA}"
u8"Ω"	u8"\xE2\x84\xA6"	u8"\u2126"	u8"\U00002126"	u8"\N{OHM SIGN}"

Raw	\x	\u	\U	\N
u8"A"	u8"\x41"	u8"\u0391"	u8"\U00000391"	u8"\N{LATIN CAPITAL LETTER A}"
u8"A"	u8"\xCE\x91"	u8"\u0391"	u8"\U00000391"	u8"\N{GREEK CAPITAL LETTER ALPHA}"
u8"A"	u8"\xD0\x90"	u8"\uD090"	u8"\U0000D090"	u8"\N{CYRILLIC CAPITAL LETTER A}"
u8"A"	u8"\xE1\x8E\xAA"	u8"\u13AA"	u8"\U000013AA"	u8"\N{CHEROKEE LETTER GO}"
u8"A"	u8"\xEA\x93\xAE"	u8"\uA4EE"	u8"\U0000A4EE"	u8"\N{LISU LETTER A}"
u8"A"	u8"\xF0\x90\x8A\xA0"	n/a	u8"\U000102A0"	u8"\N{CARIAN LETTER A}"
u8"A"	u8"\xF0\x96\xBD\x80"	n/a	u8"\U00016F40"	u8"\N{MIAO LETTER ZZYA}"

Names

Each Unicode character has a Name property associated. They correspond to the names in the English edition of ISO/IEC 10646. These names are meant to be used as unique identifiers for each character. Once assigned they never change. Such names contain only characters from a limited set: the capital letters A-Z, the digits 0-9, spaces, and hyphens; all of them are in the C++ basic character set. Supporting these names is the bare minimum necessary for this feature.

However, for added convenience, some leeway in the matching of these names is preferable. The names use only capital letters, but it would be convenient to ignore case so that the following all result in the same string:

```
"\N{LATIN CAPITAL LETTER A}" // exact match
"\N{Latin capital letter A}"
"\N{latin capital letter a}"
```

Another convenient feature would be to allow hyphens and spaces to be treated the same, so as to make the following result in the same string:

```
"\N{ZERO WIDTH SPACE}" // exact match
"\N{ZERO-WIDTH SPACE}" // common spelling, e.g. Wikipedia
"\N{zero-width space}"
```

The Unicode Standard describes which transformations are guaranteed to still produce unique identifiers and recommends the loose matching rule UAX44-LM2 (https://www.unicode.org/reports/tr44/#UAX44-LM2): "Ignore case, whitespace, underscore ('_'), and all medial hyphens except the hyphen in U+1180 HANGUL JUNGSEONG O-E". This is the most liberal set of transformations that can be forwards-compatible. Stricter sets of transformations are also possible. Python 3 allows lowercasing the names, but no other differences; Perl 6 allows only exact matching names.

Name Aliases

In addition to the Name property, Unicode characters can also have several values of the Name_Alias property assigned to them. The values of this property are effectively additional names for a character. Aliases can be corrections, names for characters that have no Name property, alternative names, or common abbreviations.

Assignment of these aliases follows the same rules of assignment of Names: they draw from the same character set, once assigned the values are immutable, and they share the same namespace. Thanks to this last property, it is possible to allow named character escape sequences to match via both the Name and Name_Alias property.

```
"\N{NO-BREAK SPACE}"      // matches Name for U+00A0
"\N{KANNADA LETTER LLLA}" // matches correction alias for U+0CDE
"\N{NBSP}"                // matches abbreviation alias for for U+00A0
"\N{LINE FEED}"           // matches control character alias for U+000A
"\N{LF}"                  // matches abbreviation alias for U+000A
```

All of Python 3, Perl 5, and Perl 6 support name aliases in their named character escapes.

These aliases are defined normatively by the Unicode Standard, but not all of them are normative in ISO/IEC 10646. This means that, without a normative reference to the Unicode Standard, only names or correction aliases two can be specified to work.

Named Character Sequences

In addition to names and aliases, there is a third set of identifiers that shares the same namespace and the same rules of assignment: named character sequences. These names represent sequences of Unicode characters for which there is a need for an identifier; they are used e.g. to correlate with identifiers in other standards. Again, because these share the same namespace, it is possible to add these as a third option for matching named character escape sequences without conflict.

```
"\N{LATIN SMALL LETTER I WITH MACRON AND GRAVE}"  // matches the named sequence
<U+012B, U+0300>
"\N{KEYCAP DIGIT ZERO}"                           // matches the named sequence
<U+0030, U+FE0F, U+20E3>"
```

Python 3 does not support named character sequences, but Perl 5 and Perl 6 do.

Proposal

This paper proposes a new character escape sequence that enables the programmer to refer to a character by one of its names. Specifically, out of the possibilities delineated above, this paper proposes the following kinds of name matching:

1. Allow matching characters by the Name property value;
2. Allow matching characters by any type of Name_Alias property values (correction, control, alternate, figment, and abbreviation);
3. Allow matching any of those values in a case-insensitive manner.

This specific set of features was chosen by virtue of being both reasonably simple and reasonably convenient. A larger feature set could also be specified if there is strong feedback in that direction.

Note that, unlike with the `\u` and `\U` syntaxes, which are allowed in identifiers, the proposed `\N` is only valid in character and string literals.

Bikeshedding

The syntax used in the description above uses `\N{NAME}` as the escape sequence for this feature merely because of the author's familiarity with Python 3. Perl 5 uses the same syntax as Python 3 but Perl 6 uses `\c[NAME]`, instead. Either of these two syntaxes would generate no significant conflicts with present day C++ syntax. The only potential conflict that

this could cause would be with code that has character and string literals with extraneous backslashes: presently, the string literal `"\N{NBSP}"` is equivalent to `"N{NBSP}"`. This doesn't seem like something that occurs often and in the unlikely case that it occurs, it's easy to search for and to work around (one can simply remove the backslash).

Other alternatives are also possible, as long as we paint the shed red.

Technical Specifications

Add the following item after 2 [intro.refs], item (1.2):

> (1.3) — The Unicode Consortium. The Unicode Standard. http://www.unicode.org/versions/latest/ (http://www.unicode.org/versions/latest/)

Edit 5.13.3 [lex.ccon] as follows:

> *escape-sequence*:
>
> *simple-escape-sequence*
>
> *octal-escape-sequence*
>
> *hexadecimal-escape-sequence*
>
> *named-escape-sequence*
>
> *named-escape-sequence*:
>
> `\N{` *n-char-sequence* `}`
>
> *n-char-sequence*:
>
> *n-char*
>
> *n-char-sequence n-char*
>
> *n-char*: one of
>
> `A B C D E F G H I J K L M N O P Q R S T U V W X Y Z`
>
> `a b c d e f g h i j k l m n o p q r s t u v w x y z`
>
> `0 1 2 3 4 5 6 7 8 9`
>
> `-`, or space

Add the following paragraph after 5.13.3 [lex.ccon], paragraph 9:

[10] A *named-escape-sequence* designates the character whose Unicode Name or Name_Alias property value is equal to the *n-char-sequence* after replacing all instances of the lowercase letters with their corresponding uppercase letters. If there is no such character, the program is ill-formed.

Edit 5.13.5 [lex.string], paragraph 15 as follows:

[15] Escape sequences and *universal-character-names* in non-raw string literals have the same meaning as in character literals, except that the single quote `'` is representable either by itself or by the escape sequence `\'`, and the double quote `"` shall be preceded by a `\`, and except that a *universal-character-name* or *named-escape-sequence* in a `char16_t` string literal may yield a surrogate pair. In a narrow string literal, a *universal-character-name* or *named-escape-sequence* may map to more than one char element due to multibyte encoding. The size of a `char32_t` or wide string literal is the total number of escape sequences, *universal-character-names*, *named-escape-sequences*, and other characters, plus one for the terminating `U'\0'` or `L'\0'`. The size of a `char16_t` string literal is the total number of escape sequences, *universal-character-names*, *named-escape-sequences*, and other characters, plus one for each character requiring a surrogate pair, plus one for the terminating `u'\0'`. [*Note:* The size of a `char16_t` string literal is the number of code units, not the number of characters. — *end note*] Within `char32_t` and `char16_t` string literals, any universal-character-names shall be within the range `0x0` to `0x10FFFF`. The size of a narrow string literal is the total number of escape sequences and other characters, plus at least one for the multibyte encoding of each *universal-character-name* or *named-escape-sequence*, plus one for the terminating `'\0'`.

Edit 5.2 [lex.phases], paragraph 1, step 5 as follows:

5. Each source character set member in a character literal or a string literal as well as each escape sequence ~~and~~, *universal-character-name*, and *named-escape-sequence* in a character literal or a non-raw string literal, is converted to the corresponding member of the execution character set ([lex.ccon], [lex.string]); if there is no corresponding member, it is converted to an implementation-defined member other than the null (wide) character.